

STANDOUT PROJECTS • SPEAKER DECK

Rohith Shabad

The five projects I lead with — and how to talk about them.

Speaker scripts live in the .pptx Notes pane; cue cards in the companion PDF.

AGENDA

The five standouts

01 CommandCore

Agent orchestration

One chat that routes intent to specialized AI agents.

02 Beacon

Content automation

An AI news-video pipeline that ships itself daily.

03 Skills Hub

Force-multiplier

25 reusable skills wired into my AI command line.

04 Financial Intelligence

Live data products

Dashboards that turn market filings into signals.

05 Rent-tracker

Production automation

A bank-scraping bot that tracks who paid rent.

CommandCore

“Imagine one chat window that understands what you want — and quietly dispatches the right specialist to do it.”

What it does

A control center where you type a request and it routes that intent to the right specialized AI agent — each with its own tools and memory. Telegram + inline keyboards, zero learning curve.

How it works

A router reads intent and hands off to an agent. State is persisted in a database, so it's restart-safe — reboot and it resumes exactly where it left off.

Why it's hard

Orchestration: typed tools, persisted memory, and getting multiple agents to cooperate without stepping on each other. That separates a system from a demo.

Code → github.com/rohith547/commandcore

[Node/TS](#)[Prisma](#)[OpenAI](#)[Telegram](#)[Agents](#)[Docker](#)

Beacon

“Every morning a fresh AI news video gets made — and I barely touch it.”

What it does

An end-to-end content pipeline: fetch the day's stories, verify them, write the script, generate a voice, render the video, publish. The one human step is a single approval tap.

How it works

Stages run in sequence — fetch, verify, script, voice, video, publish — each checkpointed so a failure never burns the whole run. Cost is capped per run.

Why it's hard

Trust. LLMs hallucinate, so I fact-check BEFORE generating, checkpoint every stage, and keep a human at the one decision that matters — the publish.

Code → github.com/rohith547/beacon

Python

OpenAI

TTS

ffmpeg

cron

Human-in-loop

Skills Hub

“Here's how I build 75 apps without reinventing the wheel 75 times.”

What it does

25 reusable engineering 'skills' — my best patterns — distilled from across my projects, plus a browsable React app to explore them, with tests.

How it works

Each skill is a documented folder wired into my AI command line, so every new build automatically follows my proven conventions instead of starting from scratch.

Why it's hard

Compounding leverage: every project makes the next one faster. It externalizes how I think, not just what I shipped — a system that builds systems.

Code → github.com/rohith547/skills-hub

React

TypeScript

Vite

Vitest

CLI

Single-source

Financial Intelligence

“I turn raw market filings into signals a person can act on in seconds.”

What it does

A family of live dashboards: smart-money-13f tracks what institutions buy, insider-tracker watches executives buying their own stock, macro-pulse scores the economy, fed-watch follows policy.

How it works

Each pulls live financial data, normalizes messy filings, and renders a glanceable view — every component handles loading, error, empty and populated states.

Why it's hard

Financial data is messy and noisy. The value is distilling it into something trustworthy and decision-ready a human reads in seconds.

Code → github.com/rohith547/smart-money-13f

[React](#)[TypeScript](#)[Tailwind](#)[Live data](#)[Dashboards](#)[Vercel](#)

Rent-tracker

“I had a boring monthly question — who actually paid rent? — so I automated the whole answer.”

What it does

A Telegram bot logs into the bank, reads transactions, fuzzy-matches each deposit to the right tenant, and reports ‘7 of 7 tenants paid’ every cycle. No spreadsheets.

How it works

It scrapes the bank, matches deposits to tenants by name and amount, and pings me on Telegram — counting unique tenants, not raw transactions.

Why it's hard

The bank fights automation — sessions expire instantly headless. Solved with fresh visible-browser logins, careful session handling, and idempotent runs so it never double-counts.

Code → private repo — live walkthrough on request

Python

Playwright

Telegram

Fuzzy-match

Idempotent

Production

The through-line

Five projects, one engineer, one consistent way of working.

Range + depth

Agents, automation, tooling, live-data products, real-world reliability.

Systems, not demos

Restart-safe, idempotent, checkpointed, tested — built to run unattended.

AI as a multiplier

LLMs inside real products, with verification, cost caps and a human where it counts.

So what? Point me at a problem and I'll turn it into a dependable, AI-powered product — fast, tested, and in production.